02471 Machine Learning for Signal Processing

Exercise Solutions

Problem 2.1

Consider the case where you have a discrete time signal, x_n of length N, where n is the time index, and a linear FIR filter of length L, with the coefficients, θ_l.

Ordinarily, the output would be calculated as a convolution between the input signal and filter weights, but it can also be described as a matrix product, $y = X\theta$. Describe exactly how you would construct X and θ in order to compute y.

Solution:

The standard filtering operation using a Finite Impulse Response (FIR) filter is defined by the discrete convolution sum. The output y_n at time step n is computed as:

$$y_n = \sum_{l=0}^{L-1} \theta_l x_{n-l} \tag{1}$$

where θ_l are the filter coefficients for $l = 0, 1, \ldots, L-1$.

To express this operation as a matrix-vector product $y = X\theta$, we structure the filter coefficients as a column vector and the input signal as a matrix where each row represents the time-delayed samples of the input signal interacting with the filter.

1. Constructing the Parameter Vector θ

The filter coefficients are arranged into an $L \times 1$ column vector θ:

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{L-1} \end{bmatrix} \tag{2}$$

2. Constructing the Input Matrix X

To satisfy the convolution sum, each row of the matrix X must contain the current and past values of the signal x_n, shifted by one time step per column. This forms a **Toeplitz matrix**.

Assuming we compute the ”valid” convolution (where we only compute outputs for time indices where the filter fully overlaps with the available data, i.e., $n \in [L-1, N-1]$), X is constructed as an $(N - L + 1) \times L$ matrix:

$$X = \begin{bmatrix} x_{L-1} & x_{L-2} & \cdots & x_0 \\ x_L & x_{L-1} & \cdots & x_1 \\ x_{L+1} & x_L & \cdots & x_2 \\ \vdots & \vdots & \ddots & \vdots \\ x_{N-1} & x_{N-2} & \cdots & x_{N-L} \end{bmatrix} \tag{3}$$

If we assume a ”full” convolution (where we assume zero-padding for x_n outside the range $0 \leq n < N$), the matrix X would simply be expanded with zeros at the boundaries, resulting in an $(N + L - 1) \times L$ matrix:

$$X_{full} = \begin{bmatrix} x_0 & 0 & \dots & 0 \\ x_1 & x_0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ x_{N-1} & x_{N-2} & \dots & x_{N-L} \\ 0 & x_{N-1} & \dots & x_{N-L+1} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & x_{N-1} \end{bmatrix} \tag{4}$$

3. Computing the Output y

With X and θ defined, the filtering operation is executed via a simple matrix multiplication, yielding the output column vector y:

$$y = X\theta = \begin{bmatrix} y_{L-1} \\ y_L \\ \vdots \\ y_{N-1} \end{bmatrix} \tag{5}$$

Each element of y corresponds exactly to the dot product of a row in X and the vector θ, thus flawlessly replicating the convolution sum.

Problem 2.2

Consider the estimation task of a parameter to a model, $\hat{\theta} = f(y, X)$, where y is the output vector, X is our input matrix, θ_o is the true parameter and $\hat{\theta}$ denotes the random variable of the associated estimator.

First show that the mean-square error, MSE $= \mathbb{E}[(\hat{\theta} - \theta_o)^2]$ can be decomposed into the bias and the variance.

Next, argue whether a biased or unbiased estimator is best suited for minimizing the MSE. Include formulas as you see necessary to support your analysis.

Solution:

1. Bias-Variance Decomposition of MSE

We begin with the definition of the Mean Squared Error (MSE) and add/subtract the expected value of the estimator, $\mathbb{E}[\hat{\theta}]$, inside the square:

$$\begin{aligned} \text{MSE} &= \mathbb{E}\left[(\hat{\theta} - \theta_o)^2\right] \\ &= \mathbb{E}\left[(\hat{\theta} - \mathbb{E}[\hat{\theta}] + \mathbb{E}[\hat{\theta}] - \theta_o)^2\right] \end{aligned} \tag{6}$$

Expanding the square yields three terms:

$$\text{MSE} = \mathbb{E}\left[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2\right] + \mathbb{E}\left[(\mathbb{E}[\hat{\theta}] - \theta_o)^2\right] + 2\mathbb{E}\left[(\hat{\theta} - \mathbb{E}[\hat{\theta}])(\mathbb{E}[\hat{\theta}] - \theta_o)\right] \tag{7}$$

Let’s analyze each term individually:

- **Variance**: The first term is the standard definition of the estimator's variance: $\text{Var}(\hat{\theta}) = \mathbb{E}\left[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2\right]$.
- **Bias Squared**: The second term consists of non-random constants (since the expectation over $\hat{\theta}$ is already evaluated). Thus, $\mathbb{E}\left[(\mathbb{E}[\hat{\theta}] - \theta_o)^2\right] = (\mathbb{E}[\hat{\theta}] - \theta_o)^2 = \text{Bias}(\hat{\theta})^2$.
- **Cross-term**: The term $(\mathbb{E}[\hat{\theta}] - \theta_o)$ is a constant with respect to the outer expectation and can be pulled out: $2(\mathbb{E}[\hat{\theta}] - \theta_o) \cdot \mathbb{E}[\hat{\theta} - \mathbb{E}[\hat{\theta}]]$. Because $\mathbb{E}[\hat{\theta} - \mathbb{E}[\hat{\theta}]] = \mathbb{E}[\hat{\theta}] - \mathbb{E}[\hat{\theta}] = 0$, the entire cross-term evaluates to zero.

Summing these parts, we arrive at the standard decomposition:

$$\text{MSE} = \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2 \tag{8}$$

2. Minimizing the MSE (Biased vs. Unbiased)

An **unbiased estimator** inherently satisfies $\text{Bias}(\hat{\theta}) = 0$, reducing its MSE entirely to the estimator's variance: $\text{MSE}_{unbiased} = \text{Var}(\hat{\theta})$.

While zero bias intuitively seems ideal, a **biased estimator** is often better suited for minimizing the overall MSE. In many models (such as Ridge Regression or shrinkage estimators), introducing a small, controlled amount of bias acts as a regularization mechanism. This restricts the model's sensitivity to noisy data, which can drastically reduce its variance.

If the reduction in variance exceeds the penalty of the squared bias introduced, the overall MSE will decrease:

$$\text{Var}_{biased}(\hat{\theta}) + \text{Bias}_{biased}(\hat{\theta})^2 < \text{Var}_{unbiased}(\hat{\theta}) \tag{9}$$

Because the MSE is a sum of strictly non-negative terms, allowing a non-zero bias allows us to exploit this *bias-variance tradeoff* to achieve a lower total error than would be mathematically possible if we strictly enforced $\text{Bias}(\hat{\theta}) = 0$.

Problem 2.3

Show that any function that is a norm is a convex function.

Solution:
A function f defined on a convex set $\mathcal{C}$ is convex if, for all $\theta_1, \theta_2 \in \mathcal{C}$ and all $\lambda \in [0, 1]$,

$$f\big(\lambda\theta_1 + (1-\lambda)\theta_2\big) \leq \lambda f(\theta_1) + (1-\lambda) f(\theta_2). \tag{10}$$

The domain $\mathbb{R}^l$ of a norm is a convex set. Let $\|\cdot\|_p$ satisfy the three norm properties and take $\theta_1, \theta_2 \in \mathbb{R}^l$ and $\lambda \in [0, 1]$. Then

$$\begin{aligned}
\|\lambda\theta_1 + (1-\lambda)\theta_2\|_p &\leq \|\lambda\theta_1\|_p + \|(1-\lambda)\theta_2\|_p && \text{(property 3, triangle inequality)} \\
&= |\lambda| \, \|\theta_1\|_p + |1-\lambda| \, \|\theta_2\|_p && \text{(property 2, homogeneity)} \\
&= \lambda\|\theta_1\|_p + (1-\lambda)\|\theta_2\|_p && \text{(since } \lambda,\ 1-\lambda \geq 0\text{)}.
\end{aligned}$$

This is exactly the definition of convexity, so every norm is convex. Property 1 is not needed for this argument. ■

Problem 2.4

The signal $\mathsf{x}_n = a\mathsf{x}_{n-1} + \mathsf{v}_n$ is an AR(1) process with $|a| < 1$ and white noise v_n of variance σ_v^2. The signal $\mathsf{y}_n = \mathsf{x}_n + b$ is created with $\mathbb{E}[\mathsf{x}_n] = 0$. Determine $r_y(k)$ and compute $r_y(-2)$ for $a = 0.8$, $b = 0.5$, $\sigma_v^2 = 1$.

Solution:

1. Autocorrelation of the AR(1) process

Since $|a| < 1$ the process is wide-sense stationary. The noise v_n is white and x_{n-k} depends only on v_m with $m \leq n-k$. Hence $\mathbb{E}[\mathsf{v}_n\mathsf{x}_{n-k}] = 0$ for $k \geq 1$, and $\mathbb{E}[\mathsf{v}_n\mathsf{x}_n] = \sigma_v^2$.

For $k = 0$:

$$r_x(0) = \mathbb{E}[\mathsf{x}_n(a\mathsf{x}_{n-1} + \mathsf{v}_n)] = a\, r_x(1) + \sigma_v^2. \tag{11}$$

For $k \geq 1$:

$$r_x(k) = \mathbb{E}[(a\mathsf{x}_{n-1} + \mathsf{v}_n)\mathsf{x}_{n-k}] = a\, r_x(k-1) \quad \Rightarrow \quad r_x(k) = a^k r_x(0). \tag{12}$$

Inserting $r_x(1) = a\, r_x(0)$ in the first equation gives $r_x(0) = a^2 r_x(0) + \sigma_v^2$, i.e.

$$r_x(0) = \frac{\sigma_v^2}{1 - a^2}. \tag{13}$$

Because $r_x(-k) = r_x(k)$, we obtain for all k

$$r_x(k) = \frac{\sigma_v^2}{1 - a^2}\, a^{|k|}. \tag{14}$$

2. Autocorrelation of $\mathsf{y}_n = \mathsf{x}_n + b$

Using $\mathbb{E}[\mathsf{x}_n] = 0$,

$$\begin{aligned} r_y(k) &= \mathbb{E}[(\mathsf{x}_n + b)(\mathsf{x}_{n-k} + b)] = r_x(k) + b\,\mathbb{E}[\mathsf{x}_n] + b\,\mathbb{E}[\mathsf{x}_{n-k}] + b^2 \\ &= \frac{\sigma_v^2}{1 - a^2}\, a^{|k|} + b^2. \end{aligned} \tag{15}$$

The offset adds the constant b^2 (the squared mean) at every lag.

3. Numerical value

For $a = 0.8$, $b = 0.5$, $\sigma_v^2 = 1$:

$$r_y(-2) = \frac{1}{1 - 0.64}\, 0.8^2 + 0.25 = \frac{0.64}{0.36} + 0.25 = \frac{16}{9} + \frac{1}{4} = \frac{73}{36} \approx 2.03. \tag{16}$$

Problem 2.5

Explain the desired and error signal of a filter setup. Estimate a Wiener filter that removes the noise w_n from $x_n = s_n + w_n$ ($f_s = 8$ kHz), argue for the filter length, plot the frequency responses of the signals and the filter and evaluate the result.

Solution:

1. Desired and error signal

In a filter setup the filter (coefficients θ) acts on the input x_n and produces the output $\hat{d}_n = \sum_{l=0}^{L-1} \theta_l x_{n-l} = \theta^\top \boldsymbol{x}_n$. The **desired signal** d_n is the signal we would like the output to reproduce, and the **error signal** is the difference

$$e_n = d_n - \hat{d}_n = d_n - \theta^\top \boldsymbol{x}_n. \tag{17}$$

The filter is chosen to minimize $\mathbb{E}[e_n^2]$. In this problem the input is $x_n = s_n + w_n$, the desired signal is the clean signal $d_n = s_n$, and the error is $e_n = s_n - \hat{s}_n$, which is the remaining noise plus the distortion of the signal.

2. Wiener solution

Setting the gradient of $J(\theta) = \mathbb{E}[(d_n - \theta^\top \boldsymbol{x}_n)^2]$ to zero gives the normal equations

$$\Sigma_x \, \theta_* = p, \qquad [\Sigma_x]_{ij} = r_x(i-j), \qquad p_k = \mathbb{E}[d_n x_{n-k}] = r_{dx}(k). \tag{18}$$

Since s_n and w_n are uncorrelated, $\Sigma_x = \Sigma_s + \Sigma_w$ and $p_k = r_s(k)$. In the frequency domain, for $L \to \infty$ (non-causal), the solution is

$$H(f) = \frac{S_s(f)}{S_s(f) + S_w(f)}, \tag{19}$$

which passes frequencies where the signal dominates and attenuates those where the noise dominates. In practice $r_x(k)$ and $r_{dx}(k)$ are replaced by the (biased) sample estimates $\hat{r}(k) = \frac{1}{N}\sum_n x_n x_{n-k}$, and the Toeplitz system is solved (`solve_toeplitz`). Since s_n and w_n are both available, $d_n = s_n$ can be used directly to estimate p.

3. Choice of filter length

The filter must be long enough to resolve the spectral structure of $S_s/(S_s+S_w)$: a longer filter has a finer frequency resolution and approximates the ideal response better, so the MSE decreases with L. On the other hand, Σ_x and p are estimated from a finite number of samples, and the number of parameters grows with L (larger variance, over-fitting; cf. Problem 2.2). I therefore estimated the filter on the first half of the data and measured the MSE on the second half, for L between 1 and 512. I chose the smallest L whose held-out MSE is within 2 % of the best one (Fig. 1). This gives

$$L = 39 \tag{20}$$

(held-out MSE -19.7 dB, versus -19.8 dB for the best value $L = 114$). A short filter keeps the computation and estimation variance low at practically no loss in MSE.

4. Results and evaluation

Using all samples to estimate the filter with $L = 39$, the signal-to-noise ratio is improved from 3.2 dB at the input to 8.9 dB at the output, a gain of 5.7 dB. Fig. 2 shows the power spectral densities (Welch) of s_n, w_n, x_n and $\hat{s}_n$ (left), and the frequency response of the Wiener filter together with the ideal non-causal response $S_s/(S_s + S_w)$ (right).

The filter should approximate the ideal response: a gain close to 0 dB where the signal spectrum lies above the noise spectrum, and strong attenuation where the noise dominates (compare the two curves in the right panel). The remaining difference to the ideal response comes from the finite length and the finite data. Where signal and noise overlap in frequency, the filter cannot separate them, and it trades noise reduction against signal distortion; this is the

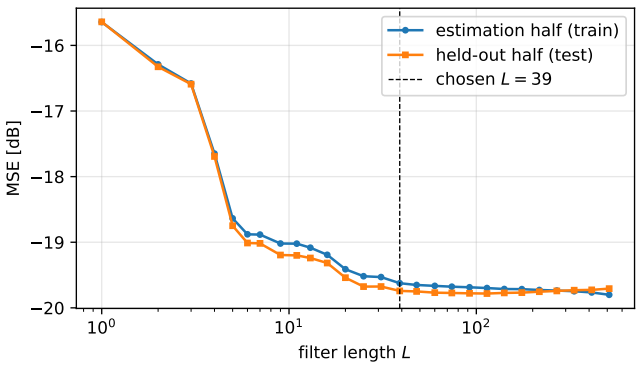


Figure 1: MSE as a function of the filter length L, on the data used for the estimation and on held-out data.

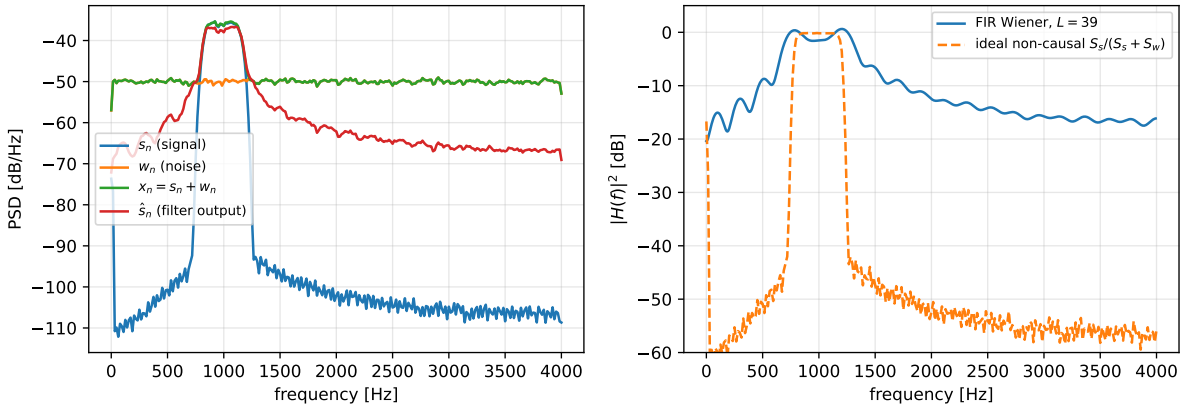


Figure 2: Left: spectra of the signal, noise, input and filter output. Right: frequency response of the FIR Wiener filter and the ideal response.

origin of the remaining error (the output SNR is limited by the spectral overlap, not only by the filter length).

Problem 2.6

Conduct a time-frequency analysis of `problem2_6.wav` by plotting the spectrogram, argue for the window choices, relate the time between the beats to the spectrogram, and identify the spectral signature of the different “instruments”.

Solution:

1. Choice of spectrogram parameters

The signal is sampled at $f_s = 22050\,\text{Hz}$ and lasts $4.04\,\text{s}$. The short-time Fourier transform is subject to a time/frequency resolution trade-off: a window of N_w samples gives a frequency resolution of about f_s/N_w and a time resolution of about N_w/f_s.

- **Window length** $N_w = 1024$ (46 ms, $\Delta f = 21.5$ Hz). The resolution of ≈ 20 Hz is fine enough to separate the low-frequency drum (tens to a few hundred Hz) from drums with energy at higher frequencies. The ≈ 46 ms window is short compared to the time between the beats (see below), so successive beats do not smear into each other, but long enough to give a good frequency resolution for the decaying drum sounds.
- **Window function**: Hann. It has low side lobes, so the strong low-frequency components do not leak into (and mask) the weak high-frequency content, at the price of a slightly wider main lobe than the rectangular window.
- **Overlap** 87.5 % (hop 128 samples = 5.8 ms). With a Hann window, at least 50 % overlap is needed to avoid losing information at the window edges; the larger overlap gives a smooth time axis and a precise localization of the beats.

2. Beats and spectral signatures

The beats were found as peaks of the spectral flux (the sum over frequency of the positive increases in the log-magnitude of the spectrogram, bottom of Fig. 3). 17 beats are detected, and the median time between consecutive beats is 0.238 s, corresponding to a tempo of about 252 beats per minute. The 87.5 % overlap gives a time resolution of 5.8 ms in the localization of the beats. For each beat, the spectrum in the ≈ 46 ms after the onset was summarized as the energy in 12 logarithmically spaced bands, and the beats were grouped into three classes using k-means ($k = 3$ was chosen from the three distinct spectral shapes that are visible by eye in the spectrogram and by listening). The classes are marked with colors in Fig. 3 and their mean spectra are shown in Fig. 4.

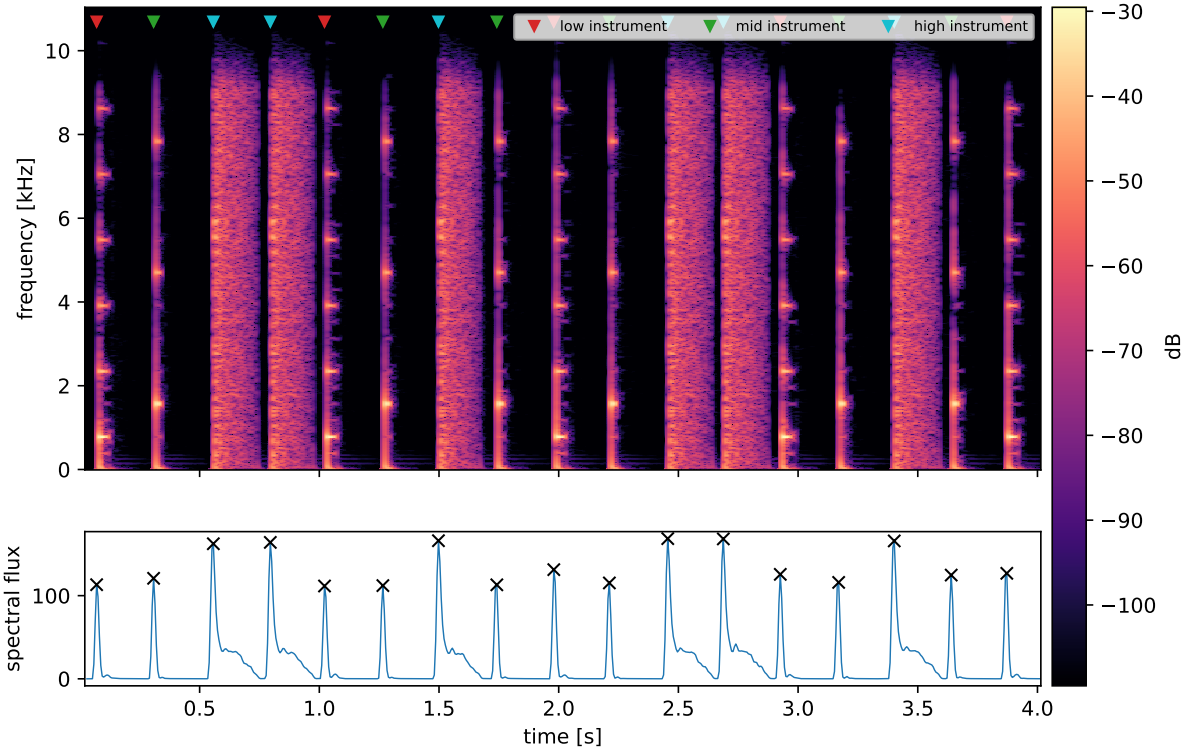


Figure 3: Spectrogram (Hann window, 1024 samples, 87.5 % overlap) with detected beats (triangles, colored by instrument class) and the spectral-flux onset function.

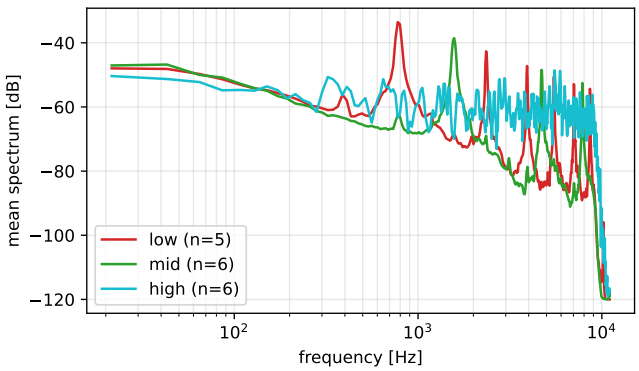


Figure 4: Mean spectrum of the beats in each class.

Class	Number of beats	Spectral centroid [Hz]	Peak frequency [Hz]
low	5	2253	775
mid	6	2544	1572
high	6	4179	5534

Table 1: Spectral signature of the instrument classes (sorted from low to high frequency content).

The three “instruments” are distinguished by where their energy is located, as quantified by the spectral centroid and peak frequency in the table: the class with the lowest centroid corresponds to a bass-drum type sound, the middle class to a snare type sound, and the class with the highest centroid to a hi-hat type sound. Their repetition pattern is visible directly in the spectrogram as vertical structures that recur with the beat period.

Problem 2.7

Identify the signals in the noise-cancelling setup, describe the principle, explain why the source must be uncorrelated with the noise, and implement LMS, NLMS and RLS noise cancellers.

Solution:

1. The signals in the setup

- **source signal**: the speech s_n that we want to recover. It reaches only the primary microphone.
- h: the unknown linear system (acoustic path / impulse response) that the noise passes through from the noise source to the primary microphone. We model it as an FIR filter.
- **primary microphone** signal $p_n = s_n + (h * v)_n$: the *desired signal* d_n of the adaptive filter. It contains the speech plus filtered noise.
- d: the **reference** (noise-only) microphone signal v_n, i.e. the *input* x_n of the adaptive filter. It is a measurement of the noise source before it passes through h.
- w: the **adaptive FIR filter**, which should approximate h.

- $\boldsymbol{a}$: the **filter output** $\hat{d}_n = \sum_l w_l v_{n-l}$, an estimate of the noise component $(h * v)_n$ in the primary signal.
- $\boldsymbol{b}$: the **error signal** $e_n = p_n - a_n$. It is also the *output of the whole system*: the primary signal with the estimated noise removed, i.e. the estimate $\hat{s}_n$ of the speech.
- $\boldsymbol{c}$: the **update of the filter coefficients**, computed by the adaptive algorithm from the error e_n (and the reference regressor) and fed back to w.

2. Principle and the need for uncorrelated source and noise

The adaptive algorithm minimizes the power of the error $e_n = p_n - a_n$, with $n_n := (h * v)_n$ denoting the noise at the primary microphone:

$$\mathbb{E}[e_n^2] = \mathbb{E}\big[(s_n + n_n - a_n)^2\big] = \mathbb{E}[s_n^2] + \mathbb{E}\big[(n_n - a_n)^2\big] + 2\,\mathbb{E}\big[s_n(n_n - a_n)\big]. \tag{21}$$

The filter output a_n is a linear combination of the reference v_{n-l}, so $n_n - a_n$ is a function of the noise reference only. If the speech is uncorrelated with the noise, the cross-term vanishes and

$$\mathbb{E}[e_n^2] = \mathbb{E}[s_n^2] + \mathbb{E}\big[(n_n - a_n)^2\big]. \tag{22}$$

The speech power $\mathbb{E}[s_n^2]$ does not depend on w. Hence minimizing the output power is the same as minimizing the *noise residual* $\mathbb{E}[(n_n - a_n)^2]$, which is minimized (equal to zero) for $w = h$, and then $e_n = s_n$. The system therefore needs no knowledge of s_n or of h.

If the speech is correlated with the reference (e.g. speech leaking into the reference microphone), the cross-term is non-zero, and the filter reduces the output power by also cancelling parts of the speech itself; the output is then a distorted speech signal. This is demonstrated numerically below.

3. Experimental setup

The speech signal (32000 samples, 8 kHz) was scaled to a peak value of 0.5. The reference noise v_n is white Gaussian noise. It is filtered through each of the three impulse responses (low-pass, high-pass, band-pass; lengths: low-pass: 11, high-pass: 9, band-pass: 13), and its variance is chosen such that the noise at the primary microphone has the same power as the speech, i.e. an input SNR of 0 dB. This is a demanding case in which the benefit of the canceller is easy to assess both by the SNR and by listening. All adaptive filters have length $M =$ length of h (the true model order, so exact cancellation is possible; a shorter filter would leave a residual and a longer one would add misadjustment noise). The adaptive filters are

$$\begin{aligned}
\text{LMS:}\quad & w_{n+1} = w_n + \mu\, e_n \boldsymbol{v}_n, \\
\text{NLMS:}\quad & w_{n+1} = w_n + \frac{\tilde{\mu}}{\epsilon + \|\boldsymbol{v}_n\|^2}\, e_n \boldsymbol{v}_n, \\
\text{RLS:}\quad & k_n = \frac{P_{n-1}\boldsymbol{v}_n}{\lambda + \boldsymbol{v}_n^\top P_{n-1}\boldsymbol{v}_n}, \quad w_n = w_{n-1} + k_n e_n, \quad P_n = \lambda^{-1}\big(P_{n-1} - k_n \boldsymbol{v}_n^\top P_{n-1}\big),
\end{aligned}$$

with $\boldsymbol{v}_n = [v_n, \ldots, v_{n-M+1}]^\top$ and $e_n = p_n - w^\top \boldsymbol{v}_n$ (a priori error). The step size of LMS must satisfy $0 < \mu < 2/\lambda_{\max}(\Sigma_v)$ for convergence in the mean ($\lambda_{\max} \le \operatorname{tr}\Sigma_v = M\sigma_v^2$, so $\mu < 2/(M\sigma_v^2)$ is a safe bound); NLMS removes the dependence on the input power, with $0 < \tilde{\mu} < 2$; RLS uses a forgetting factor $\lambda \in (0, 1]$ and $P_0 = \delta^{-1} I$ with a small δ. The step size, $\tilde{\mu}$ and λ were selected from a grid by the output SNR in the last half of the signal (Fig. 5).

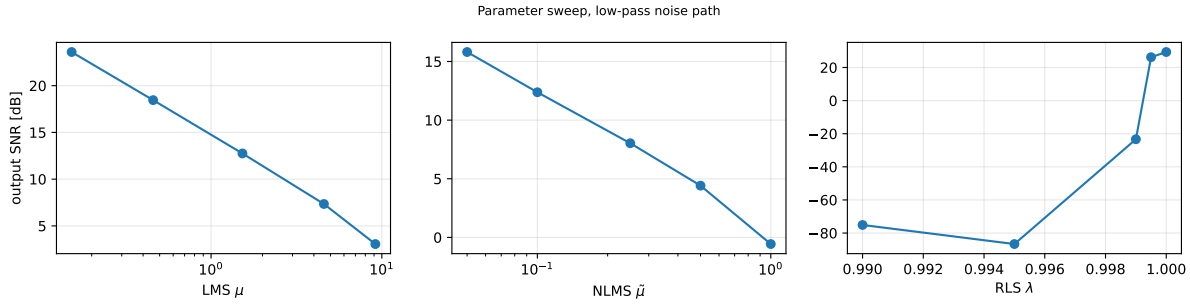


Figure 5: Output SNR as a function of the algorithm parameter (shown for the first noise path).

4. Results

The performance is measured by the output SNR, $10 \log_{10} \sum s_n^2 / \sum (e_n - s_n)^2$, over the last half of the signal (after convergence), and by the misalignment $10 \log_{10} \|w - h\|^2 / \|h\|^2$ of the final filter.

Noise path	Algorithm	Parameter	SNR in [dB]	SNR out [dB]	Misalignment [dB]
low-pass	LMS	0.153	0.0	23.6	21.5
low-pass	NLMS	0.05	0.0	15.8	21.5
low-pass	RLS	1	0.0	29.3	21.5
high-pass	LMS	0.229	-0.0	23.9	22.5
high-pass	NLMS	0.05	-0.0	16.0	22.5
high-pass	RLS	1	-0.0	30.8	22.5
band-pass	LMS	0.201	0.0	24.0	23.6
band-pass	NLMS	0.05	0.0	16.2	23.6
band-pass	RLS	1	0.0	27.7	23.6

Table 2: Performance of the adaptive noise cancellers (parameter: μ for LMS, $\tilde{\mu}$ for NLMS, λ for RLS).

5. Discussion

- **Convergence speed.** RLS whitens the input through $P_n \approx \Sigma_v^{-1}/n$ and converges in the order of M to $2M$ samples independently of the eigenvalue spread, at the cost of $O(M^2)$ operations per sample. LMS and NLMS cost $O(M)$ and converge more slowly (Fig. 6); LMS depends on the input power through μ, which NLMS removes by normalization, so NLMS is easier to tune (Fig. 5).
- **Steady state.** The choice of μ ($\tilde{\mu}$, λ) is a trade-off: a large step size gives fast convergence and tracking but a large excess error (misadjustment, here also caused by the speech acting as measurement noise in the error), a small step size gives low misadjustment but slow convergence. This is the optimum seen in the parameter sweep. With $\lambda = 1$ RLS has the lowest misadjustment for a stationary path; $\lambda < 1$ is required if h changes in time.
- **Filter estimates.** Fig. 7 compares the estimated filters with the true response. The filters can only be identified in frequency regions where the reference has power; since the reference is white, the entire band is excited for all three paths, and the misalignment in the table quantifies the remaining error.

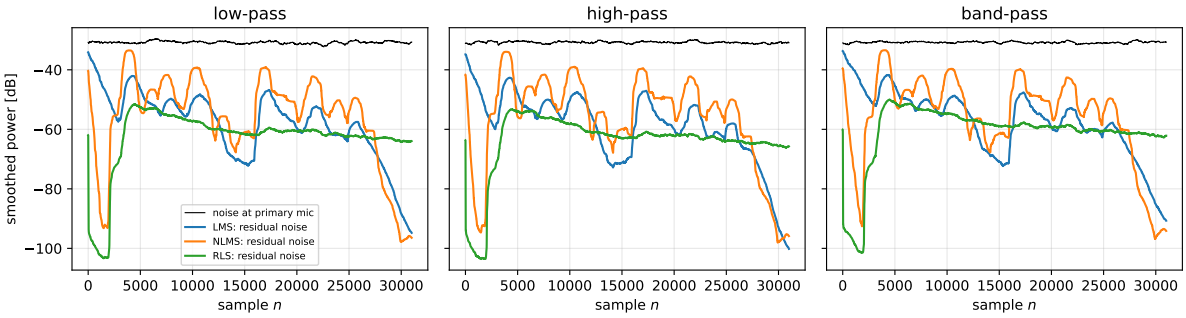


Figure 6: Learning curves: smoothed power of the noise at the primary microphone and of the residual noise $e_n - s_n$ for the three algorithms.

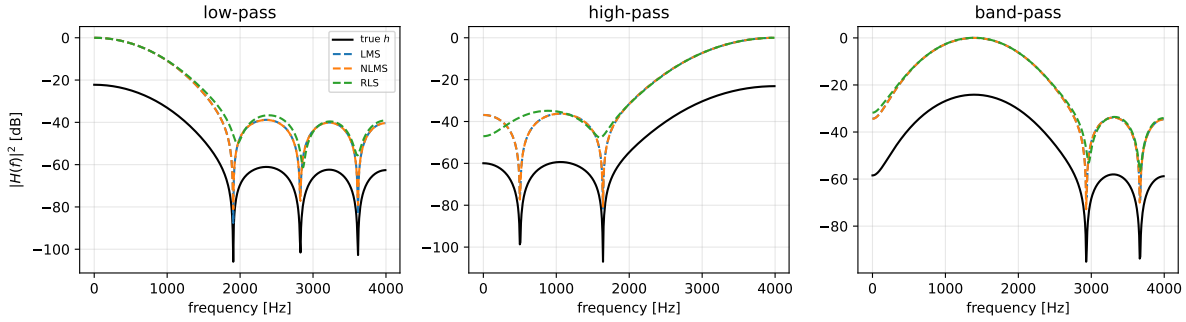


Figure 7: Frequency response of the true noise path and of the estimated adaptive filters.

- **Speech leakage into the reference.** Adding a fraction α of the (power-normalized) speech to the reference signal and using NLMS ($\tilde{\mu} = 0.5$) with the band-pass noise path gives the output SNRs in Table 3. The performance degrades as α grows, since the filter learns to cancel part of the speech, confirming why the source and noise must be uncorrelated.

α	SNR out [dB]
0.0	4.5
0.1	3.4
0.3	1.4

Table 3: NLMS output SNR when a fraction α of the speech leaks into the reference microphone.

For subjective evaluation, the noisy and the cleaned signals are saved as audio files (`audio/noisy_*.wav`, `audio/clean_*_*.wav`) by the script in the appendix. Comparing them by ear complements the SNR figures in the table above.

Appendix: Python code for Problems 2.5–2.7

```
"""
02471 Machine Learning for Signal Processing -- Problem set 2, problems 2.5-2.7.

Usage:   python solutions.py [data_dir] [out_dir]
          data_dir (default ./data) must contain
              problem2_5_signal.mat, problem2_5_noise.mat,
              problem2_6.wav,
              problem2_7_speech.mat, problem2_7_lpir.mat,
              problem2_7_hpir.mat,  problem2_7_bpir.mat
Output:  out_dir/figs/*.pdf   figures used by exercise_solutions.tex
          out_dir/results.tex  numbers used by exercise_solutions.tex
          out_dir/audio/*.wav  noisy / cleaned speech for the listening test
"""
import sys
from pathlib import Path

import numpy as np
import scipy.io as sio
import scipy.signal as sg
from scipy.cluster.vq import kmeans2
from scipy.io import wavfile
from scipy.linalg import solve_toeplitz

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

DATA = Path(sys.argv[1]) if len(sys.argv) > 1 else Path("data")
OUT = Path(sys.argv[2]) if len(sys.argv) > 2 else Path(".")
FIGS, AUDIO = OUT / "figs", OUT / "audio"
FIGS.mkdir(parents=True, exist_ok=True)
AUDIO.mkdir(parents=True, exist_ok=True)
FS = 8000
RNG = np.random.default_rng(0)
R = []  # lines of results.tex


def macro(name, value):
    R.append(f"\\newcommand{{\\{name}}}{{{value}}}")


def load_mat(name):
    """Load the (single) numeric variable of a .mat file as a 1-D float array."""
    m = sio.loadmat(DATA / name)
    arrs = [np.asarray(v) for k, v in m.items()
            if not k.startswith("__") and np.issubdtype(np.asarray(v).dtype, np.number)]
    return max(arrs, key=lambda a: a.size).astype(float).ravel()


def db(x):
    return 10 * np.log10(x)


# ---------------------------------------------------------------------------
# Problem 2.5  FIR Wiener filter
# ---------------------------------------------------------------------------
def xcorr(a, b, L):
    """Biased estimate of E[a_n b_{n-k}], k = 0..L-1."""
    N = len(a)
    return np.array([a[k:] @ b[:N - k] for k in range(L)]) / N


def wiener_fir(x, d, L):
    """Solve Sigma_x theta = p, Sigma_x = Toeplitz(r_x), p_k = E[d_n x_{n-k}]."""
    return solve_toeplitz(xcorr(x, x, L), xcorr(d, x, L))


def problem_2_5():
    s, w = load_mat("problem2_5_signal.mat"), load_mat("problem2_5_noise.mat")
    n = min(len(s), len(w))
```

```
    s, w = s[:n], w[:n]
    x = s + w
    half = n // 2                       # estimate on first half, test on second
    Ls = np.unique(np.round(np.geomspace(1, 512, 30)).astype(int))
    mse_tr, mse_te = [], []
    for L in Ls:
        th = wiener_fir(x[:half], s[:half], L)
        sh = sg.lfilter(th, 1, x)
        mse_tr.append(np.mean((s[:half] - sh[:half]) ** 2))
        mse_te.append(np.mean((s[half:] - sh[half:]) ** 2))
    mse_tr, mse_te = np.array(mse_tr), np.array(mse_te)
    # smallest L whose held-out MSE is within 2 % of the best held-out MSE
    L_opt = int(Ls[np.argmax(mse_te <= 1.02 * mse_te.min())])

    th = wiener_fir(x, s, L_opt)        # final filter on all data
    sh = sg.lfilter(th, 1, x)
    snr_in = db(np.sum(s ** 2) / np.sum(w ** 2))
    snr_out = db(np.sum(s ** 2) / np.sum((s - sh) ** 2))

    fig, ax = plt.subplots(figsize=(5.5, 3.3))
    ax.semilogx(Ls, db(mse_tr), "o-", ms=3, label="estimation␣half␣(train)")
    ax.semilogx(Ls, db(mse_te), "s-", ms=3, label="held-out␣half␣(test)")
    ax.axvline(L_opt, color="k", ls="--", lw=.8, label=f"chosen␣$L={L_opt}$")
    ax.set_xlabel("filter␣length␣$L$"); ax.set_ylabel("MSE␣[dB]"); ax.legend()
    ax.grid(alpha=.3); fig.tight_layout(); fig.savefig(FIGS / "p25_mse_vs_L.pdf")

    nper = 512
    f, Ps = sg.welch(s, FS, nperseg=nper); _, Pw = sg.welch(w, FS, nperseg=nper)
    _, Px = sg.welch(x, FS, nperseg=nper); _, Pe = sg.welch(sh, FS, nperseg=nper)
    fh, H = sg.freqz(th, 1, worN=512, fs=FS)
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.6))
    for P, lab in [(Ps, "$s_n$␣(signal)"), (Pw, "$w_n$␣(noise)"),
                   (Px, "$x_n=s_n+w_n$"), (Pe, r"$\hat␣s_n$␣(filter␣output)")]:
        ax[0].plot(f, db(P), label=lab)
    ax[0].set_xlabel("frequency␣[Hz]"); ax[0].set_ylabel("PSD␣[dB/Hz]")
    ax[0].legend(fontsize=8); ax[0].grid(alpha=.3)
    ax[1].plot(fh, db(np.abs(H) ** 2 + 1e-12), label=f"FIR␣Wiener,␣$L={L_opt}$")
    ax[1].plot(f, db(Ps / (Ps + Pw)), "--", label=r"ideal␣non-causal␣$S_s/(S_s+S_w)$")
    ax[1].set_ylim(-60, 5)
    ax[1].set_xlabel("frequency␣[Hz]"); ax[1].set_ylabel(r"$|H(f)|^2$␣[dB]")
    ax[1].legend(fontsize=8); ax[1].grid(alpha=.3)
    fig.tight_layout(); fig.savefig(FIGS / "p25_freq.pdf")

    macro("resWienerL", L_opt)
    macro("resWienerSNRin", f"{snr_in:.1f}")
    macro("resWienerSNRout", f"{snr_out:.1f}")
    macro("resWienerGain", f"{snr_out␣-␣snr_in:.1f}")
    macro("resWienerLtestMSE", f"{db(mse_te[Ls␣==␣L_opt][0]):.1f}")
    macro("resWienerMinMSE", f"{db(mse_te.min()):.1f}")
    macro("resWienerLmin", int(Ls[np.argmin(mse_te)]))


# ---------------------------------------------------------------------------
# Problem 2.6  Spectrogram of audio
# ---------------------------------------------------------------------------
def problem_2_6():
    fs, y = wavfile.read(DATA / "problem2_6.wav")
    y = y.astype(float)
    if y.ndim > 1:
        y = y.mean(axis=1)
    y /= np.max(np.abs(y))
    nper, hop = 1024, 128
    f, t, S = sg.spectrogram(y, fs, window="hann", nperseg=nper,
                             noverlap=nper - hop, mode="magnitude")
    SdB = 20 * np.log10(S + 1e-10)

    # onset detection: spectral flux of log-compressed magnitude
    C = np.log1p(100 * S / S.max())
    flux = np.maximum(np.diff(C, axis=1), 0).sum(axis=0)
    flux = np.convolve(flux, np.hanning(5) / np.hanning(5).sum(), "same")
    min_dist = int(0.06 * fs / hop)
    pk, _ = sg.find_peaks(flux, height=flux.mean() + 1.0 * flux.std(), distance=min_dist)
    onsets = t[pk + 1]
```

```
    ioi = np.diff(onsets)

    # per-onset spectral signature: mean log-spectrum in 12 log-spaced bands
    edges = np.geomspace(40, fs / 2, 13)
    feats, spec = [], []
    for i in pk + 1:
        seg = S[:, i:i + 8].mean(axis=1)
        spec.append(seg)
        b = [seg[(f >= lo) & (f < hi)].sum() ** 2 for lo, hi in zip(edges[:-1], edges[1:])]
        b = db(np.array(b) + 1e-12)
        feats.append(b - b.max())
    feats, spec = np.array(feats), np.array(spec)
    K = 3
    _, lab = kmeans2(feats, K, minit="++", seed=0)
    cent = np.array([np.sum(f * spec[lab == k].mean(0)) / np.sum(spec[lab == k].mean(0))
                     if np.any(lab == k) else np.nan for k in range(K)])
    order = np.argsort(cent)                        # low -> high
    rank = {int(k): r for r, k in enumerate(order)}
    lab = np.array([rank[int(l)] for l in lab])
    names = ["low", "mid", "high"]
    cols = ["tab:red", "tab:green", "tab:cyan"]

    fig, ax = plt.subplots(2, 1, figsize=(10, 6), sharex=True,
                           gridspec_kw={"height_ratios": [3, 1]})
    im = ax[0].pcolormesh(t, f / 1e3, SdB, shading="auto", cmap="magma",
                          vmin=SdB.max() - 80, vmax=SdB.max())
    for r in range(K):
        ax[0].plot(onsets[lab == r], np.full((lab == r).sum(), f[-1] / 1e3 * .97), "v",
                   color=cols[r], ms=5, label=f"{names[r]} instrument")
    ax[0].set_ylabel("frequency [kHz]"); ax[0].legend(loc="upper right", fontsize=7, ncol=3)
    fig.colorbar(im, ax=ax, label="dB", pad=0.01)
    ax[1].plot(t[1:], flux, lw=.7); ax[1].plot(onsets, flux[pk], "kx")
    ax[1].set_xlabel("time [s]"); ax[1].set_ylabel("spectral flux")
    fig.savefig(FIGS / "p26_spectrogram.pdf", bbox_inches="tight")

    fig, ax = plt.subplots(figsize=(5.5, 3.3))
    for r in range(K):
        if np.any(lab == r):
            ax.semilogx(f[1:], db(spec[lab == r].mean(0)[1:] ** 2 + 1e-12),
                        color=cols[r], label=f"{names[r]} (n={np.sum(lab == r)})")
    ax.set_xlabel("frequency [Hz]"); ax.set_ylabel("mean spectrum [dB]")
    ax.grid(alpha=.3, which="both"); ax.legend(); fig.tight_layout()
    fig.savefig(FIGS / "p26_signatures.pdf")

    rows = []
    for r in range(K):
        m = lab == r
        if not m.any():
            continue
        ms = spec[m].mean(0)
        rows.append(f"{names[r]} & {m.sum()} & {np.sum(f * ms) / np.sum(ms):.0f} & "
                    f"{f[1:][np.argmax(ms[1:])]:.0f} \\\\")
    med = np.median(ioi)
    macro("resSixFs", fs)
    macro("resSixDur", f"{len(y) / fs:.2f}")
    macro("resSixDf", f"{fs / nper:.1f}")
    macro("resSixWinMs", f"{1e3 * nper / fs:.0f}")
    macro("resSixHopMs", f"{1e3 * hop / fs:.1f}")
    macro("resSixNons", len(onsets))
    macro("resSixIOI", f"{med:.3f}")
    macro("resSixBPM", f"{60 / med:.0f}")
    macro("resSixRows", " ".join(rows))


# ---------------------------------------------------------------------------
# Problem 2.7  Adaptive noise cancelling: LMS, NLMS, RLS
# ---------------------------------------------------------------------------
def regressors(d, M):
    """Row n = [d_n, d_{n-1}, ..., d_{n-M+1}] (zero padded)."""
    dp = np.concatenate([np.zeros(M - 1), d])
    return np.lib.stride_tricks.sliding_window_view(dp, M)[:, ::-1]


```

```
def lms(d, p, M, mu):
    U, w, e = regressors(d, M), np.zeros(M), np.zeros(len(d))
    for n in range(len(d)):
        e[n] = p[n] - w @ U[n]
        w += mu * e[n] * U[n]
    return e, w


def nlms(d, p, M, mu, eps=1e-6):
    U, w, e = regressors(d, M), np.zeros(M), np.zeros(len(d))
    for n in range(len(d)):
        e[n] = p[n] - w @ U[n]
        w += mu * e[n] * U[n] / (eps + U[n] @ U[n])
    return e, w


def rls(d, p, M, lam, delta=0.01):
    U, w, e = regressors(d, M), np.zeros(M), np.zeros(len(d))
    P = np.eye(M) / (delta * np.var(d))
    for n in range(len(d)):
        u = U[n]
        Pu = P @ u
        k = Pu / (lam + u @ Pu)
        e[n] = p[n] - w @ u
        w += k * e[n]
        P = (P - np.outer(k, Pu)) / lam
    return e, w


def snr(s, resid):
    return db(np.sum(s ** 2) / np.sum(resid ** 2))


def smooth_db(x, N=1000):
    return db(np.convolve(x ** 2, np.ones(N) / N, "valid") + 1e-14)


def problem_2_7():
    s = load_mat("problem2_7_speech.mat")
    s = s / np.max(np.abs(s)) * 0.5
    N = len(s)
    irs = {k: load_mat(f"problem2_7_{k}ir.mat") for k in ("lp", "hp", "bp")}
    names = {"lp": "low-pass", "hp": "high-pass", "bp": "band-pass"}
    SNR_IN = 0.0                                        # dB at the primary microphone
    v = RNG.standard_normal(N)                          # reference noise (unit variance)
    results, curves = [], {}
    rows = []
    sweep_rows = {}

    for key, h in irs.items():
        noise = sg.lfilter(h, 1, v)
        g = np.sqrt(np.sum(s ** 2) / np.sum(noise ** 2) * 10 ** (-SNR_IN / 10))
        d = g * v                                       # reference microphone
        nz = g * noise                                  # noise at primary microphone
        p = s + nz                                      # primary microphone
        M = len(h)
        wtrue = h                                       # noise at primary = h * d, so the ideal
              w is h
        s2d = np.var(d)
        wavfile.write(AUDIO / f"noisy_{key}.wav", FS, (p / np.abs(p).max() * .9 * 32767).
            astype(np.int16))

        # step-size / forgetting-factor sweeps (final SNR over the last half)
        def final_snr(e):
            return snr(s[N // 2:], (e - s)[N // 2:])
        mu_l = np.array([.01, .03, .1, .3, .6]) * 2 / (M * s2d) * 0.5
        mu_n = np.array([.01, .02, .05, .1, .25, .5, 1.0])
        lam_r = np.array([.99, .995, .999, .9995, 1.0])
        sw_l = [final_snr(lms(d, p, M, m)[0]) for m in mu_l]
        sw_n = [final_snr(nlms(d, p, M, m)[0]) for m in mu_n]
        sw_r = [final_snr(rls(d, p, M, l)[0]) for l in lam_r]
        best = dict(LMS=(mu_l[int(np.argmax(sw_l))], lms),
                    NLMS=(mu_n[int(np.argmax(sw_n))], nlms),
```

```
                    RLS=(lam_r[int(np.argmax(sw_r))], rls))
        sweep_rows[key] = (mu_l, sw_l, mu_n, sw_n, lam_r, sw_r)

        curves[key] = {}
        for alg, (par, fn) in best.items():
            e, w = fn(d, p, M, par)
            curves[key][alg] = (e, w)
            mis = db(np.sum((w - wtrue) ** 2) / np.sum(wtrue ** 2))
            rows.append(f"{names[key]} & {alg} & {par:.3g} & {snr(s, p - s):.1f} & "
                        f"{snr(s[N//2:], (e - s)[N//2:]):.1f} & {mis:.1f} \\\\")
            wavfile.write(AUDIO / f"clean_{key}_{alg}.wav", FS,
                          (e / np.abs(p).max() * .9 * 32767).astype(np.int16))
        curves[key]["_noise"] = nz
        curves[key]["_p"] = p
        curves[key]["_d"] = d
        curves[key]["_h"] = wtrue

    # --- figures ------------------------------------------------------------
    fig, ax = plt.subplots(1, 3, figsize=(12, 3.4), sharey=True)
    for a, key in zip(ax, irs):
        nz = curves[key]["_noise"]
        a.plot(smooth_db(nz), "k", lw=.8, label="noise at primary mic")
        for alg in ("LMS", "NLMS", "RLS"):
            e = curves[key][alg][0]
            a.plot(smooth_db(e - s), label=f"{alg}: residual noise")
        a.set_title(names[key]); a.set_xlabel("sample $n$"); a.grid(alpha=.3)
    ax[0].set_ylabel("smoothed power [dB]"); ax[0].legend(fontsize=7)
    fig.tight_layout(); fig.savefig(FIGS / "p27_learning.pdf")

    fig, ax = plt.subplots(1, 3, figsize=(12, 3.4))
    for a, key in zip(ax, irs):
        M = len(irs[key])
        fh, Ht = sg.freqz(curves[key]["_h"], 1, worN=512, fs=FS)
        a.plot(fh, db(np.abs(Ht) ** 2 + 1e-12), "k", lw=1.5, label="true $h$")
        for alg in ("LMS", "NLMS", "RLS"):
            fw, Hw = sg.freqz(curves[key][alg][1], 1, worN=512, fs=FS)
            a.plot(fw, db(np.abs(Hw) ** 2 + 1e-12), "--", label=alg)
        a.set_title(names[key]); a.set_xlabel("frequency [Hz]"); a.grid(alpha=.3)
    ax[0].set_ylabel(r"$|H(f)|^2$ [dB]"); ax[0].legend(fontsize=7)
    fig.tight_layout(); fig.savefig(FIGS / "p27_filters.pdf")

    fig, ax = plt.subplots(1, 3, figsize=(12, 3.2))
    key0 = list(irs)[0]
    mu_l, sw_l, mu_n, sw_n, lam_r, sw_r = sweep_rows[key0]
    ax[0].semilogx(mu_l, sw_l, "o-"); ax[0].set_xlabel(r"LMS $\mu$")
    ax[1].semilogx(mu_n, sw_n, "o-"); ax[1].set_xlabel(r"NLMS $\tilde\mu$")
    ax[2].plot(lam_r, sw_r, "o-"); ax[2].set_xlabel(r"RLS $\lambda$")
    ax[0].set_ylabel("output SNR [dB]")
    for a in ax:
        a.grid(alpha=.3)
    fig.suptitle(f"Parameter sweep, {names[key0]} noise path", fontsize=9)
    fig.tight_layout(); fig.savefig(FIGS / "p27_sweep.pdf")

    # --- experiment: reference microphone picks up the speech ---------------
    key = "bp"
    d, p, h = curves[key]["_d"], curves[key]["_p"], curves[key]["_h"]
    M = len(irs[key])
    leak = []
    for alpha in (0.0, 0.1, 0.3):
        dd = d + alpha * s * np.std(d) / np.std(s)
        e, _ = nlms(dd, p, M, 0.5)
        leak.append((alpha, snr(s[N // 2:], (e - s)[N // 2:])))
    macro("resLeakRows", " ".join(f"{a:.1f} & {x:.1f} \\\\" for a, x in leak))

    macro("resSevenN", N)
    macro("resSevenM", ", ".join(f"{names[k]}: {len(h)}" for k, h in irs.items()))
    macro("resSevenSNRin", f"{SNR_IN:.0f}")
    macro("resSevenRows", " ".join(rows))


if __name__ == "__main__":
    for fn in (problem_2_5, problem_2_6, problem_2_7):
        fn()
```

```
361             print("done", fn.__name__)
362         (OUT / "results.tex").write_text("\n".join(R) + "\n")
```